PASSIVE TLS FINGERPRINTING FOR CLIENT AND SERVER IDENTIFICATION

Computer Networks Project Report

Busa Bhava Ram
Roll No.: 24110083

Moram Raghavendra Sri Koushik
Roll No.: 24110210

Mukkesh Dudekula
Roll No.: 24110114

Ambati Chaitanya Ram
Roll No.: 24110035

Vunnam Thushar Chowdary
Roll No.: 24110393

I. INTRODUCTION

Transport Layer Security (TLS) protects application traffic by encrypting data exchanged between clients and servers. Although application data is encrypted, the initial TLS handshake exposes selected negotiation metadata. In particular, a ClientHello advertises protocol capabilities, cipher suites and extensions, while a ServerHello communicates the server's selected parameters. These handshake messages can therefore provide useful evidence about the software stack involved in a connection [1].

TLS fingerprinting converts selected handshake fields into compact identifiers that can be compared with known profiles. JA3 fingerprints client ClientHello messages; JA3S fingerprints server ServerHello responses. JA4 provides a newer client-fingerprinting format that normalizes selected fields, including ordering, to improve consistency for some modern clients [2]-[4]. Such fingerprints are useful for network inventory and security monitoring, but they are not cryptographic proof of application identity.

This project implements a passive, PCAP-based TLS fingerprinting workflow using Python and Scapy. The implementation extracts ClientHello and ServerHello messages, computes JA3, JA3S and JA4, and compares client fingerprints with a curated JSON reference database. Experiments cover five recognizable client profiles, additional unknown traffic, repeated connections, and a deliberate browser-profile impersonation case.

II. PROBLEM STATEMENT

The problem addressed is to identify likely TLS client or server software from observable handshake metadata without decrypting application payloads or installing an agent on the endpoint. Given a saved packet capture containing TLS traffic, the system must reconstruct relevant handshake messages, calculate fingerprints, and compare the resulting values with a reference database.

Inputs: PCAP/PCAPNG files containing TCP/TLS traffic; a JSON database of reference fingerprints and associated software metadata. Outputs: extracted ClientHello/ServerHello information, JA3/JA3S/JA4 values, and one or more matching software candidates or an “unknown” result.

Assumptions and scope: the current analyzer operates on saved captures rather than continuously monitoring a live interface. The evaluated traffic uses TLS over TCP. The implementation focuses on ClientHello-based JA3/JA4 identification and a separate JA3S extractor. JA4S, eBPF/XDP acceleration, Redis/Valkey storage, TRex traffic generation, and automated live alerting are outside the current implementation.

III. PROJECT OBJECTIVES

1. Capture representative TLS traffic passively and preserve it in PCAP/PCAPNG format for repeatable analysis.
2. Extract ClientHello and ServerHello messages, including messages that span multiple TCP segments.
3. Implement JA3 and JA3S fingerprint computation and implement JA4 as the client-side stretch fingerprint.
4. Build a reference-only fingerprint database and identify at least five distinct client profiles from observed fingerprints.
5. Evaluate repeatability, unknown fingerprints, modern-client variation, and deliberate fingerprint impersonation.
6. Document how TLS fingerprints can support monitoring while explaining false attribution and evasion limitations.

IV. FINGERPRINTING METHODOLOGY

A. TLS handshake fields

A TLS client begins the handshake with ClientHello. The message includes a legacy version field, offered cipher suites, and extensions; extensions can advertise supported versions, groups, signature algorithms, ALPN and other capabilities. The server responds with ServerHello containing the selected cipher suite and negotiated parameters [1]. In TLS 1.3, the ClientHello

legacy_version field is fixed to 0x0303 (TLS 1.2) for compatibility; the actual TLS 1.3 offer is represented through the supported_versions extension [1].

B. JA3 client fingerprint

JA3 extracts five ordered components from ClientHello: (1) TLS version field, (2) cipher-suite identifiers, (3) extension identifiers, (4) supported elliptic-curve/group identifiers, and (5) EC point formats. GREASE values are excluded. The values are serialized in the order version,ciphers,extensions,groups,point-formats, using commas between components and hyphens within a component. The resulting string is hashed with MD5 to obtain the 32-character JA3 value [2].

C. JA3S server fingerprint

JA3S applies the same general idea to ServerHello. It serializes the server version field, selected cipher suite, and ServerHello extension identifiers, then computes an MD5 digest [2]. JA3S is therefore a fingerprint of a server response in the context of a particular client offer; the response can vary with the ClientHello, so it should not be interpreted as a universal server identity by itself.

A ServerHello sample processed during development produced the JA3S string 771,4866,51-43 and MD5 digest 907bf3ecef1c987c889946b737b43de8. This illustrates the separate server-side extractor; the main multi-client evaluation table focuses on ClientHello identification.

D. JA4 client fingerprint

JA4 represents TLS-over-TCP, TLS version, SNI presence, cipher count, extension count and ALPN characteristics in a readable prefix. It then uses truncated SHA-256 hashes over normalized cipher and extension/signature-algorithm data. GREASE is ignored and cipher/extension values are sorted for the relevant hash components. This normalization can make JA4 more stable than JA3 when extension order changes, as observed in the Chrome and GnuTLS experiments [3].

E. Database matching

The analyzer compares each computed fingerprint against the reference records. A record may contain software name/version, TLS library/version, runtime details, evidence-capture filename and notes. A fingerprint can map to more than one candidate; a match therefore means “consistent with this reference profile,” not proof that the named application generated the traffic. The database contains baseline/reference observations only. Experimental observations, unknown values and impersonation results are kept as separate PCAP inputs and are not inserted merely to force a match.

V. ARCHITECTURE AND DESIGN

The design deliberately separates capture, parsing, fingerprint computation and database matching. This keeps the core analyzer small and makes experiments repeatable: packet capture tools produce files, and the Python program processes those files offline.

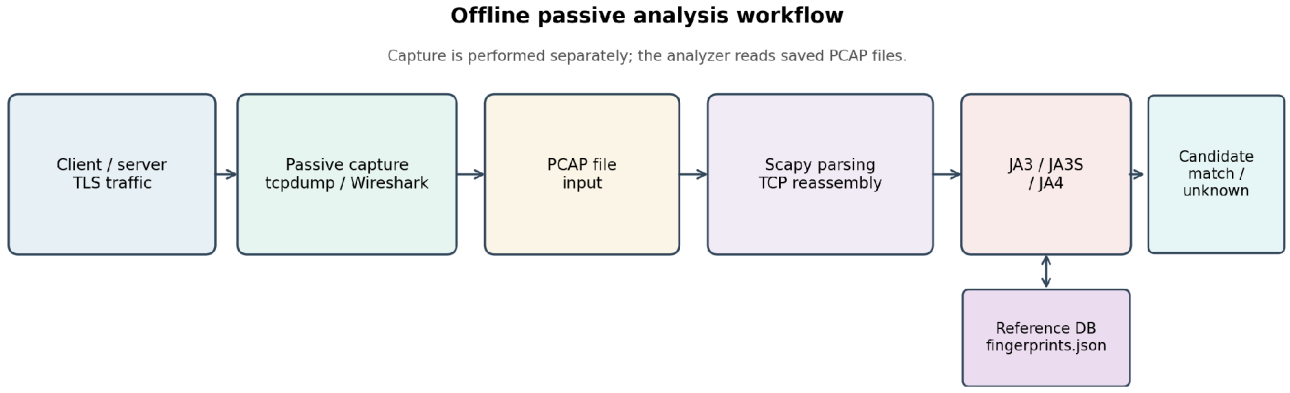


Fig. 1. Offline passive TLS fingerprinting architecture implemented in the project.

A. Components

- Traffic capture: tcpdump or Wireshark records traffic without terminating the TLS connection. PCAP/PCAPNG files are the analyzer input.
- Packet parsing and reassembly: Scapy reads captured packets. TCP session reassembly is used so a handshake message split across TCP segments can be reconstructed before TLS parsing [5].
- Fingerprint modules: src/ja3.py extracts client JA3; src/ja3s.py extracts server JA3S; src/ja4.py computes client JA4.
- Identification and reference data: src/identify.py calculates JA3/JA4 for each ClientHello and looks up data/fingerprints.json. It prints candidate metadata or an unknown result.
- Demonstration runner: test/run_wire_demo.py invokes identify.py for the selected experiment captures and labels each run.

B. Design choices and trade-offs

Python and Scapy were selected because they provide a practical packet dissection and TCP reassembly workflow without requiring a custom packet parser. JSON was selected for the small reference dataset because it is human-readable and simple to version with the source code. Offline PCAP analysis was preferred over an always-on sensor to reduce implementation complexity and simplify debugging, at the cost of not providing live alerts or continuous fleet-wide collection. eBPF/XDP, Redis/Valkey and TRex were optional technologies in the project description; they were not required for the implemented prototype and were not used.

VI. IMPLEMENTATION

A. Technology stack and modules

Item	Implementation / role
Language	Python (project virtual environment: .venv)
Packet analysis	Scapy; TCP session reassembly for segmented TLS messages
Capture / inspection	tcpdump and Wireshark; Docker network-namespace capture for the impersonation experiment
JA3 / JA3S / JA4	Custom Python modules: src/ja3.py, src/ja3s.py and src/ja4.py
Identification	src/identify.py; reads PCAP/PCAPNG and data/fingerprints.json
Database format	JSON reference records in data/fingerprints.json
Experiment automation	test/run_wire_demo.py; runs the configured capture set through identify.py

B. Processing algorithm

The core ClientHello processing path can be summarized as follows:

1. Read the PCAP/PCAPNG file and reconstruct TCP sessions so complete TLS handshake messages are available.
2. Locate each ClientHello and extract the fields required by JA3 and JA4.
3. Remove GREASE values according to the fingerprint format, serialize the relevant fields and compute the hashes.
4. Load the reference JSON database and compare the calculated JA3 and JA4 values with the stored reference values.
5. Print the packet endpoints, fingerprints, candidate software metadata, or an explicit unknown result; summarize known and unknown counts.

C. Challenges and resolutions

An initial capture inspection found packets but no parsed ClientHello or ServerHello messages. The issue was incomplete TLS handshake data at individual packet boundaries. The capture reader was updated to use Scapy TCP session reassembly, after which the test capture yielded one ClientHello and one ServerHello. GREASE handling was also important: GREASE values must be ignored for JA3/JA4 normalization rather than treated as stable client characteristics. Finally, the impersonation capture was taken inside the container network namespace because capturing only on the WSL host did not reliably expose the container traffic.

D. Validation

During development, the fingerprint functions were checked against known/reference vectors, including JA3 and JA3S examples, and GREASE handling was tested. The temporary unit-test file was subsequently removed from the final working tree to keep the demonstration repository focused; repeatable PCAP-based demonstrations are retained through the runner and standalone extractor scripts. The results in Section VIII are therefore based on the supplied experiment outputs, not on a claim that a permanent automated unit-test suite remains in the repository.

VII. TESTBED SETUP

A. Environment

Component	Recorded setup
Host environment	Windows host with WSL 2 Ubuntu
Container runtime	Docker Engine / Docker Desktop CLI version 29.3.1 (recorded during project setup)
Python environment	Project-local virtual environment: .venv; Scapy used for packet parsing

Component	Recorded setup
Capture and inspection	tcpdump, Wireshark, Docker netshoot container for shared-network-namespace capture
CPU / RAM	13th Gen Intel(R) Core(TM) i5-1340P (1.90 GHz) /16.0 GB (15.6 GB usable)
OS release	Windows 11 (26H2)/Ubuntu 24.04.4 LTS

B. Client profiles used

Profile	Recorded version / TLS stack
curl	curl 8.5.0 / OpenSSL 3.0.13
Google Chrome	156.0.8078.80 / Chromium TLS stack
GnuTLS CLI	GnuTLS 3.8.3
Go crypto/tls	Go 1.22.2
Python standard-library ssl	Python 3.12.3 / OpenSSL 3.0.13
OpenSSL CLI reference capture	OpenSSL command-line client baseline capture (version not included in this report)

C. Captures and workload

Baseline reference captures were collected for the supported client profiles and stored under captures/. Repeated-connection experiment files were stored under experiments/. The repeated experiments contain five ClientHello messages for curl, GnuTLS, Go crypto/tls and Python standard-library ssl, and two filtered example.com ClientHello messages for Chrome. The custom OpenSSL TLS experiment contains one ClientHello. The curl-impersonate experiment contains one ClientHello generated with the curl_chrome152 profile. The traffic was generated against example.com over TCP port 443; destinations varied between connections because the hostname can resolve to different service addresses.

VIII. EVALUATION

The following table summarizes the observed match counts from the six experiment files in the current demonstration set. “Known” means that the computed value matched at least one candidate in the current reference database. It does not mean that the application identity is cryptographically verified.

Experiment profile	ClientHello count	JA3 known	JA4 known	Main observation
Custom OpenSSL TLS	1	0/1	0/1	Both fingerprints unknown; no reference match.
Chrome	2	1/2	2/2	One JA3 variant was unknown; both JA4 values matched Chrome.
curl	5	5/5	5/5	All repeated ClientHellos matched the curl reference profile.
GnuTLS	5	0/5	5/5	JA3 varied across the five handshakes; JA4 matched consistently.
Go crypto/tls	5	5/5	5/5	All five repeated handshakes matched.
Python standard-library ssl	5	5/5	5/5	All five repeated handshakes matched.

Across these six experiment files, 23 ClientHello messages were processed. Five distinct reference profiles were successfully recognized by JA3 and/or JA4: curl, Chrome, GnuTLS, Go crypto/tls and Python standard-library ssl. Chrome and GnuTLS demonstrate the benefit of JA4 in the tested data: JA4 matched all observed samples, while some or all corresponding JA3 variants did not match.

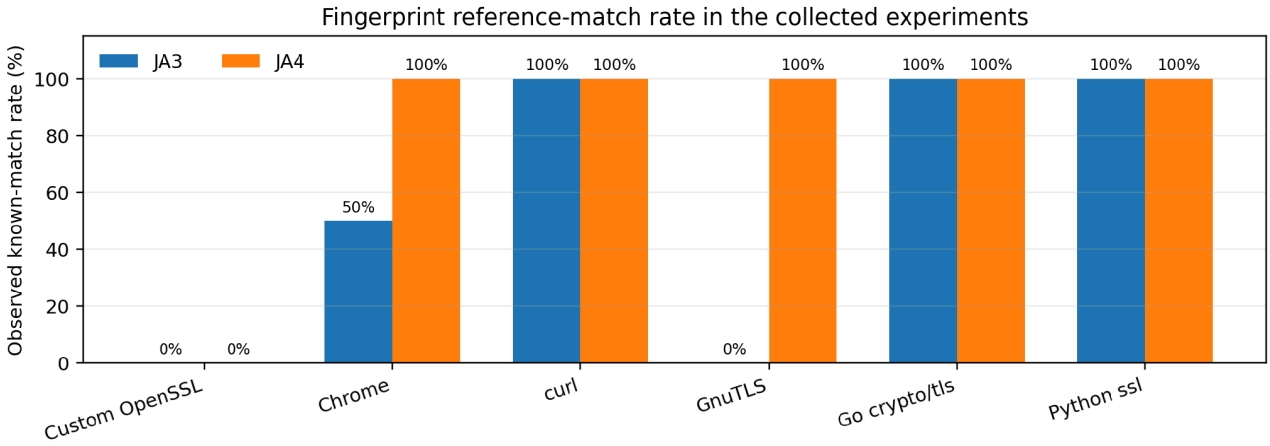


Fig. 2. Known-reference match rates for the repeated experiment captures. Percentages are based on the counts in the table above.

A. Fingerprint stability observations

- curl: five repeated ClientHello messages produced the same JA3 and JA4 values, matching the curl reference entry.
- GnuTLS: five messages produced five different JA3 strings/hashes due to changes in extension ordering, while the JA4 value remained constant and matched the reference entry.
- Chrome: the filtered capture contained two ClientHello messages with different JA3 results; one JA3 was unknown and one matched. Both JA4 values matched Chrome reference entries.
- Go crypto/tls and Python standard-library ssl: both were stable across five captured connections and matched their JA3 and JA4 reference profiles.

B. Impersonation experiment

A separate experiment used curl-impersonate with the curl_chrome152 profile. The actual process was curl-impersonate, but its ClientHello was configured to imitate a Chrome-like TLS profile. The captured ClientHello produced JA3 MD5 32bf25b02f50b9110ced3f768df1cd4e, which was unknown to the JA3 database. Its JA4 value, t13d1517h2_8daaf6152771_cb7bf5808d99, matched the Chrome JA4 reference entry. This is a concrete false-attribution demonstration: the system reports a Chrome candidate even though the known generating process was curl-impersonate. The Chrome database version is the version associated with the baseline record; the match does not establish that the impersonating process is that exact Chrome release [6].

C. Limitations

- Fingerprint spoofing: a client can deliberately construct a ClientHello resembling another application. A database match is evidence of handshake similarity, not proof of the executable or library that generated it.
- Shared fingerprints: different programs or configurations can produce identical fingerprints. The system may return candidates and cannot always identify one unique application.
- Fingerprint variation: versions, settings, extension ordering and browser randomization can change fingerprints. JA4 reduced the observed variation in the Chrome and GnuTLS experiments but does not prevent deliberate impersonation.
- Reference-database coverage: new clients and modified configurations can be reported as unknown until independently collected reference fingerprints are added.
- Visibility limits: the analyzer relies on handshake fields available in the capture. Packet loss, incomplete captures, unsupported protocols or encrypted handshake mechanisms can reduce the available information.
- Offline prototype: the current implementation analyzes saved captures and does not provide continuous production monitoring, eBPF/XDP acceleration, distributed storage or automatic alerting.

D. Security relevance

TLS fingerprints can enrich network monitoring by grouping traffic with similar client stacks, finding unusual software profiles, and helping analysts investigate potentially suspicious encrypted sessions. JA3 and JA3S can be combined with destination, timing, certificates, DNS, endpoint telemetry and behavioral evidence to investigate possible command-and-control traffic [2]. Fingerprints should be treated as one signal in a larger detection process because legitimate software can share a fingerprint and an adversary can imitate a known profile.

IX. SUMMARY

This project implemented a PCAP-based passive TLS fingerprinting prototype that extracts ClientHello and ServerHello messages and computes JA3, JA3S and JA4. A JSON reference database supports candidate matching for multiple client software profiles. Experiments on curl, Chrome, GnuTLS, Go crypto/tls and Python standard-library ssl demonstrate identification across at least five distinct profiles using JA3 and/or JA4. Repeated captures also show that JA4 was more stable than JA3 for the tested Chrome and GnuTLS traffic.

The curl-impersonate experiment demonstrates the central limitation of fingerprint-based identification: a Chrome-like JA4 match can be produced by a non-Chrome program. Therefore, the tool identifies likely handshake profiles rather than proving application identity. Future work includes JA4S, live capture and alerting, larger independently curated reference datasets, broader protocol coverage, and correlation with endpoint or flow-level telemetry.

X. REPRODUCIBILITY

Repository: https://github.com/BHAVA-RAM/CS331-T012-TLS-FINGERPRINTING. The repository should include src/, data/fingerprints.json, captures/, experiments/, test/run_wire_demo.py under code/ and the dependency/setup instructions used by the team.

A. Reproduce fingerprint extraction and identification

1. Open the repository in WSL Ubuntu and activate the project environment: source .venv/bin/activate
2. Run the configured experiment suite from the project root: python test/run_wire_demo.py
3. Analyze one selected capture directly: python src/identify.py experiments/repeat_curl.pcap
4. Analyze the impersonation capture: python src/identify.py experiments/curl_chrome152.pcap
5. Run the standalone JA3 and JA3S extractors on a capture: python src/ja3.py <capture.pcap> and python src/ja3s.py <capture.pcap>.

Use the project docs/ folder to install the tested Python and Scapy versions. For impersonation, run the curl-impersonate curl_chrome152 profile, capture its traffic from the same Docker network namespace, and analyze the resulting PCAP with identify.py. Keep the reference database unchanged.

REFERENCES

[1] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Internet Engineering Task Force, Aug. 2018. https://www.rfc-editor.org/rfc/rfc8446

[2] J. Althouse, J. Atkinson, and J. Atkins, “TLS Fingerprinting with JA3 and JA3S,” Salesforce Engineering Blog, Jan. 2018. https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/

[3] FoxIO, “JA4: TLS Client Fingerprinting - Technical Details,” JA4+ repository. https://github.com/FoxIO-LLC/ja4/blob/main/technical_details/JA4.md (accessed Nov. 6, 2026).

[4] Salesforce, “JA3 - A method for profiling SSL/TLS Clients,” source repository and algorithm notes. https://github.com/salesforce/ja3 (accessed Nov. 6, 2026).

[5] Scapy Project, “scapy.sessions - TCPSession,” Scapy documentation. https://scapy.readthedocs.io/en/latest/api/scapy.sessions.html (accessed Nov. 6, 2026).

[6] curl-impersonate, “Quick Start” and browser impersonation profiles. https://curl-impersonate.readthedocs.io/en/latest/quick_start.html (accessed Nov. 6, 2026).

APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work Distribution

#	Member Name	Roll No.	Tasks Done	Rating (1-10, unique)
1	Busa Bhava Ram	24110083	Developed the fingerprint identification pipeline, integrated the reference database and implemented JA3S extraction. prepared the final report.	10
2	Moram Raghavendra Sri Koushik	24110210	Developed the JA3 and JA4 fingerprinting modules, including TLS ClientHello parsing and fingerprint generation and prepared the final presentation.	9
3	Mukkesh Dudekula	24110114	Set up packet captures and conducted experiments using curl, Chrome, Go, GnuTLS, Python SSL and custom OpenSSL clients.	8
4	Ambati Chaitanya Ram	24110114	Performed repeated-capture validation, checked fingerprint matching results, investigated JA3 variations and tested unknown fingerprints and impersonation cases.	7
5	Vunnam Thushar Chowdary	24110393	Developed the demonstration runner, organized project documentation, coordinated the demonstration.	6

APPENDIX B: AI USAGE DETAILS

B.1 Prompts

#	AI Tool	Purpose / representative prompt
1	ChatGPT (GPT-6)	Concept explanation and implementation support: “Explain the TLS ClientHello and ServerHello fields used by JA3 and JA3S, then help implement the extraction logic in Python/Scapy.” All prompts are present in AI_USED/ folder.
2	ChatGPT (GPT-6)	Debugging packet parsing: “Why does the PCAP contain TLS packets but the parser finds no ClientHello or ServerHello, and how should TCP stream reassembly be used?” All prompts are present in AI_USED/ folder.
3	ChatGPT (GPT-6)	Experiment analysis: “Analyze the JA3 and JA4 results from the repeated curl, Chrome, GnuTLS, Go and Python ssl captures, and explain matches versus unknown fingerprints.” All prompts are present in AI_USED/ folder.
4	ChatGPT (GPT-6)	Security limitation demonstration: “Design a curl-impersonate Chrome-profile experiment to test whether our passive fingerprint matcher can mistake a Chrome-like handshake for Chrome.” All prompts are present in AI_USED/ folder.

.